

PROGRAMA ORIENTADO A OBJETOS

TRABAJO DE PROGRAMACIÓN DE SISTEMAS



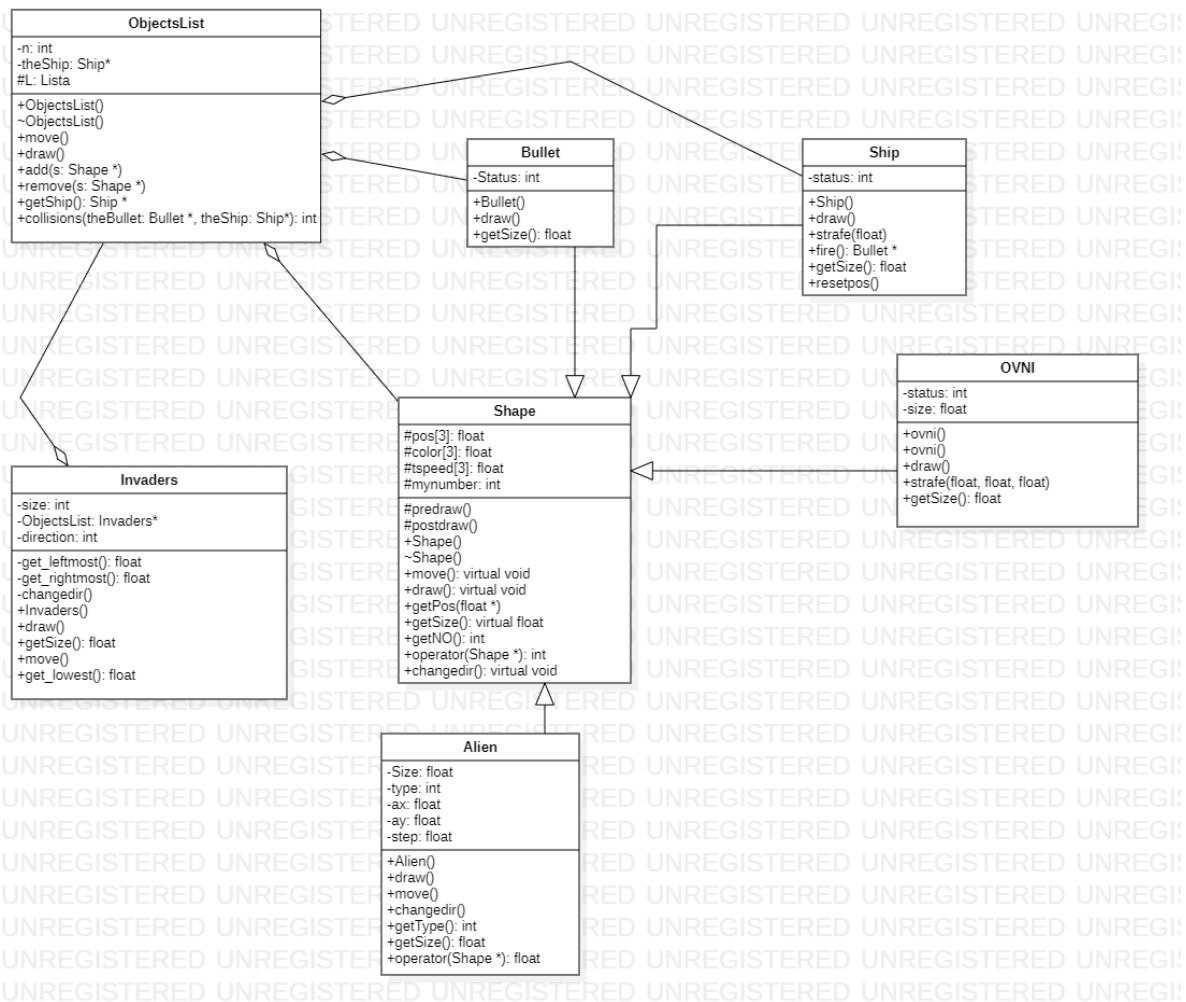
Integrantes:

Gabriela Artaza Esteban 18759
Lorena Rodríguez Carbia 18305
Pablo Lecocq Rael 15233

ÍNDICE

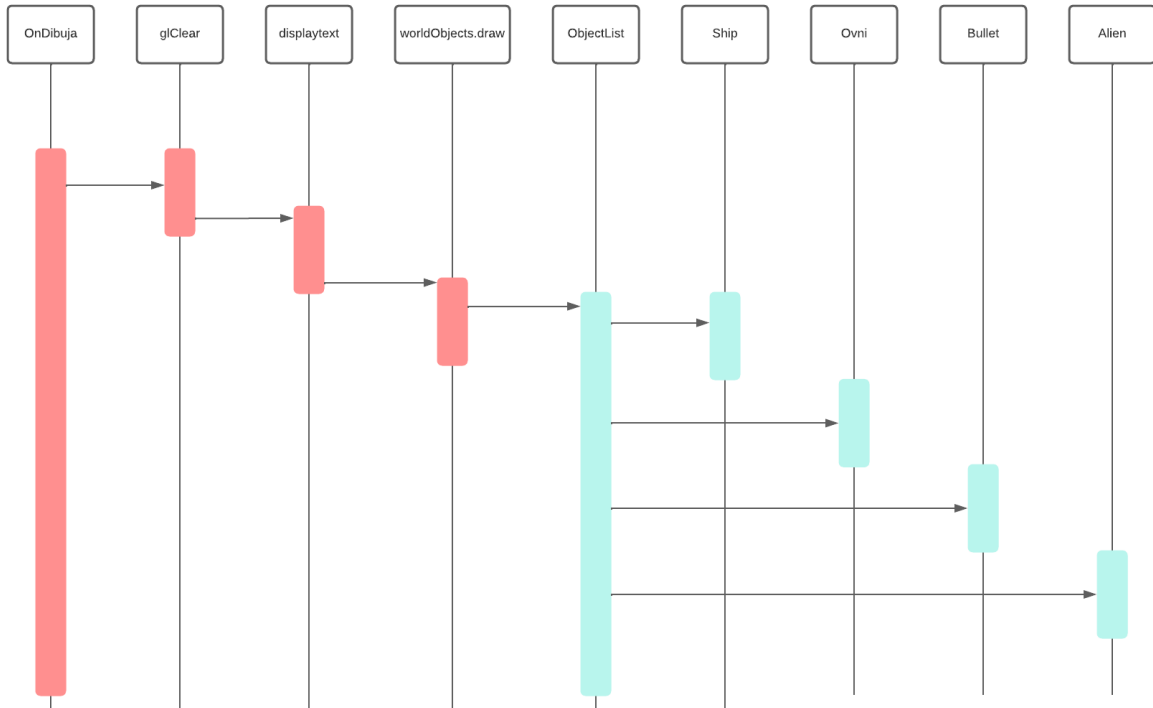
1. Diagrama de clases.....	2
2. Diagramas de interacción de las fases del juego más interesantes.....	4
3. Descripción general de funciones y estructuras de datos más representativas.....	5
4. Descripción de pruebas realizadas para probar el correcto funcionamiento del algoritmo.....	6
5. Guía de uso del programa ejecutable.....	6
6. Descripción del reparto de roles del equipo.....	7

1. Diagrama de clases

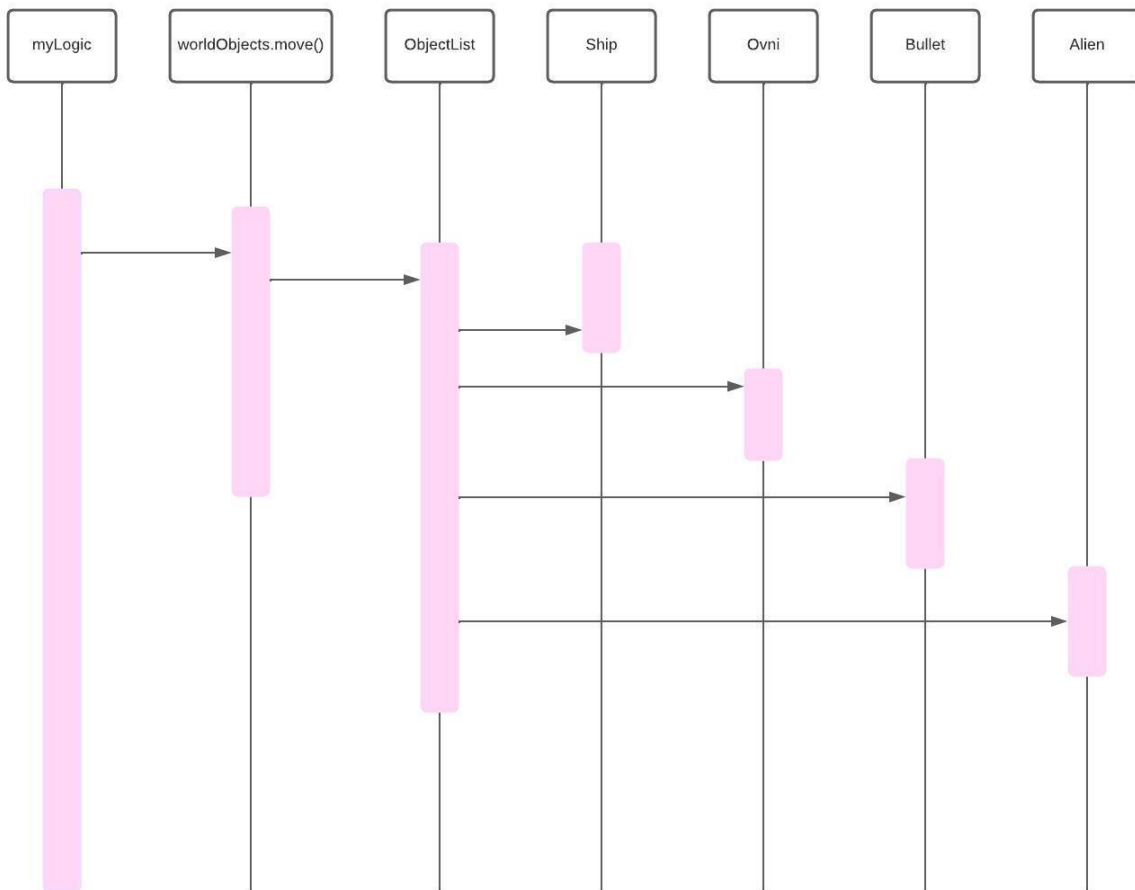


2. Diagramas de interacción de las fases del juego más interesantes

- Diagrama de secuencia **draw**:



- Diagrama de secuencia **move**:



3. Descripción general de funciones y estructuras de datos más representativas

La clase `ObjectList` maneja una lista llamada `L` donde guardamos todos los objetos que serán usados durante el resto del programa. También realiza algunas funciones básicas como dibujar, mover o eliminar los objetos cuando se detectan colisiones.

La función principal de la clase `ObjectList` es `collisions()` que devuelve un entero.

Según el entero que devuelve la lógica del juego puede saber que tipo de colisión ha ocurrido 0= no hay colisión, res=1: Proyectoil/Nave // res>=2: Alien/Proyectoil.

Las clases de los objetos del juego (`Alien`, `Ship`, `Ovni`, `Bullet`) heredan características de la clase `Shape`. Aunque cada clase tenga sus atributos particulares, al ser similares entre ellas, nos facilita la integración con el resto del código.

4. Descripción de pruebas realizadas para probar el correcto funcionamiento del algoritmo

Las pruebas realizadas en el proyecto se pueden resumir en dos grandes grupos. Por un lado, las primeras pruebas realizadas se han hecho para tratar de eliminar los errores existentes en el código original. Al solventar dichos problemas, ha surgido un problema nuevo: el juego no se ejecutaba.

Resolver este segundo problema, ha sido un trabajo bastante más complejo. En este caso, las pruebas realizadas para comprobar el funcionamiento del juego, las hemos tenido que realizar una a una. En cuanto comprobamos que una de las clases funcionaba y no contenía errores, pasamos a comprobar la siguiente, y así sucesivamente hasta realizar pruebas en cada una de las clases del juego.

5. Guía de uso del programa ejecutable

El juego consiste en destruir a todos los aliens antes de que se acerquen demasiado a la nave.

Para ganar el juego, la nave debe disparar presionando la barra espaciadora y conseguir una colisión entre el proyectil y el alien. Para conseguir destruir a todos los aliens, el avatar, es decir, la nave deberá moverse de derecha a izquierda, mediante las teclas indicadas más adelante.

El fin del juego, se consigue matando al último alien de todos, o bien si éstos se acercan demasiado a la nave y por tanto se termina el juego.

Para la utilización de nuestro programa, sólo son necesarias 3 teclas.

- Flecha derecha e izquierda para moverse en ambos sentidos
- Barra espaciadora para disparar



6. Descripción del reparto de roles del equipo

El trabajo nos lo hemos repartido de la siguiente forma:

- El trabajo que iba a realizar cada integrante se habló en conjunto en una primera reunión, concertada el día 18 de mayo.
- **Lorena** se ha encargado de realizar la *Lista.cpp*
- **Gabriela** ha implementado la clase *Ovni.h*
- **Pablo** se ha encargado de comprobar la portabilidad del programa.
- **Lorena** y **Pablo** se han encargado de realizar el método *ObjectsList.cpp* , que finalmente **Gabriela** comprobó y perfeccionó.
- **Lorena** y **Gabriela** se han encargado de hacer los *Diagramas de secuencia*
- **Gabriela** se ha encargado de hacer el *Diagrama de clases*.
- **Pablo** ha redactado la *Descripción de funciones* y **Lorena** la *Descripción de pruebas realizadas*
- Entre **todos** los participantes se puso en común todo lo realizado por separado y se probó el funcionamiento del trabajo.

7. Propuestas de mejora y valoración personal

Como una posible mejora del juego, se podrían implementar niveles, en los que cada vez que se ganase el juego, cambiase de fondo el videojuego, como simulando otra galaxia o universo.

Otra de las mejoras que se nos ha ocurrido es implementar otro objeto Ship con una forma distinta para poder escoger entre uno u otro al inicio del programa, como para poder elegir un avatar distinto.

Por último, la última mejora que se nos ha ocurrido, ha sido crear animaciones cuando ocurriesen distintas acciones en el juego. Por ejemplo, cuando un disparo destruyera un Alien, apareciese algo por pantalla.